

Module 2: Dart Programming Essentials

1. Fundamental Data Types in Dart and Their Uses

Dart is a statically typed language, which means every variable has a type.

int (Integer)

Used for whole numbers.

```
int age = 21;
```

```
int marks = 95;
```

Use:

Age

Count

IDs

Quantity

double

Used for decimal numbers.

```
double price = 99.99;
```

```
double pi = 3.14;
```

Use:

Price

Percentage

Measurement

String

Used for text.

```
String name = "Nandan";
```

```
String city = "Ahmedabad";
```

Use:

Names

Messages

Email

Labels

bool

Used for true/false values.

```
bool isLogin = true;
```

```
bool isPassed = false;
```

Use:

Login status

Conditions

Verification

List

Used to store multiple values in order.

```
List<String> fruits = ["Apple", "Banana", "Mango"];
```

Use:

Product lists

Student names

Tasks

Map

Stores data in key-value pairs.

```
Map<String, String> student = {
```

```
  "name": "Nandan",
```

```
  "city": "Rajkot"
```

```
};
```

Use:
JSON data
User profiles
Database records

dynamic
Can store any type.
dynamic value = 10;
value = "Hello";
Use:
Flexible input
Unknown type data

2. Control Structures in Dart

Control structures manage decision-making and loops.

if Statement

Executes code if condition is true.

int age = 18;

```
if (age >= 18) {  
    print("Eligible to vote");  
}
```

if-else Statement

int marks = 40;

```
if (marks >= 35) {  
    print("Pass");  
} else {  
    print("Fail");  
}
```

for Loop

Repeats code fixed number of times.

```
for (int i = 1; i <= 5; i++) {  
    print(i);  
}
```

Output:

1 2 3 4 5

while Loop

Runs while condition is true.

int i = 1;

```
while (i <= 3) {  
    print(i);  
    i++;  
}
```

switch Statement

Used for multiple conditions.

String day = "Monday";

```
switch (day) {  
    case "Monday":  
        print("Start of week");  
}
```

```

    break;
case "Sunday":
    print("Holiday");
    break;
default:
    print("Normal day");
}

```

3. Object-Oriented Programming Concepts in Dart

Dart fully supports OOP.

Class

Blueprint for objects.

```

class Student {
    String name = "Nandan";

    void display() {
        print(name);
    }
}

```

Object

```

void main() {
    Student s = Student();
    s.display();
}

```

Inheritance

One class inherits another.

```

class Animal {
    void sound() {
        print("Animal sound");
    }
}

```

```

class Dog extends Animal {
    void bark() {
        print("Bark");
    }
}

```

Polymorphism

Same method, different behavior.

```

class Animal {
    void sound() {
        print("Animal sound");
    }
}

```

```

class Dog extends Animal {
    @override
    void sound() {
        print("Bark");
    }
}

```

Interface

Dart uses classes as interfaces.

```
class Printer {  
  void printData() {}  
}
```

```
class PDFPrinter implements Printer {  
  @override  
  void printData() {  
    print("Printing PDF");  
  }  
}
```

4. Asynchronous Programming in Dart

Used for tasks like:

API calls

File download

Database fetch

Future

Represents a value available later.

```
Future<String> fetchData() {  
  return Future.delayed(  
    Duration(seconds: 2),  
    () => "Data Loaded",  
  );  
}
```

async

Marks function asynchronous.

await

Waits for Future result.

```
void main() async {  
  String data = await fetchData();  
  print(data);  
}
```

Stream

Multiple asynchronous values over time.

```
Stream<int> countStream() async* {  
  for (int i = 1; i <= 3; i++) {  
    yield i;  
  }  
}
```

Listen:

```
void main() async {  
  await for (int value in countStream()) {  
    print(value);  
  }  
}
```

Module 1 Practical:

<https://github.com/popatnandan2136/assignmentdart/tree/4d8c63e1292a9c2a66b07de72929b18921052806/lib/module1>

Module 2 Practical:

<https://github.com/popatnandan2136/assignmentdart/tree/4d8c63e1292a9c2a66b07de72929b18921052806/lib>